

IT 2012 - Unstructured Data

Lab 13 - Dashboards, Streaming, and Deployment

Assist. Prof. Dr. Dželila Mehanović

Introduction

This lab extends the Movie Industry Analytics Pipeline built in previous labs into a fully interactive, deployable web application. In Lab 12 we produced static and interactive charts as standalone files. A stakeholder who receives those files sees a single fixed view: they cannot filter by genre, zoom into a year range, or ask follow-up questions without asking you to re-run the notebook. This lab solves that problem by turning the same data and charts into a live Dash dashboard that anyone with the URL can use without installing Python. By the end of this lab the project will have a new `src/dashboard/` package, a runnable `app.py`, a `Dockerfile`, and a `docker-compose.yml`. Running two commands **`docker compose up`** will start the full stack (the Dash app and MongoDB) and serve the dashboard on **`http://localhost:8050`**.

Learning Objectives

- Build a Dash application with a proper layout using html and dcc components.
- Implement reactive callbacks that filter and re-draw Plotly charts based on user input.
- Connect dashboard callbacks to a MongoDB data source.
- Stream simulated live data into a chart using `dcc.Interval`.
- Write a production-ready Flask development server.
- Dockerfile that uses Gunicorn instead of the
- Orchestrate the Dash app and MongoDB together with `docker-`
- `compose.yml`.
- Test the containerised application locally and document the deployment
- process.

This lab introduces a new Dash-based dashboard application for the Movie Industry Analytics Pipeline. It adds a root-level Dash entry point `app.py`, a `Dockerfile` for containerizing the dashboard, a `docker-compose.yml` file for running the app together with MongoDB, and a new `src/dashboard/` package that separates the dashboard layout, callbacks, and database access logic. The lab also includes a one-time MongoDB seeding script and a submission notebook for testing and demonstrating the full dashboard workflow.

In `src/dashboard/`, you need to add the following:

- **`src/dashboard/__init__.py`** – makes the dashboard folder a Python package so its modules can be imported cleanly across the application.
- **`src/dashboard/layout.py`** – defines the full Dash page layout, including the visual structure, dashboard components, filters, charts, tables, and any other UI elements shown in the app.

- **src/dashboard/callbacks.py** – contains all Dash @callback functions that control the interactive behavior of the dashboard, such as updating charts, filtering data, responding to user input, and connecting UI components with backend logic.
- **src/dashboard/data_access.py** – contains MongoDB query helper functions used by the dashboard to retrieve, filter, and prepare movie data before it is displayed in the app.

You also need to create the following at the project root level:

- **app.py** – is the main Dash entry point at the root level. It initializes the Dash application, loads the layout, registers callbacks, and starts the dashboard server.
- **Dockerfile** – defines the container recipe for the Dash application so it can be built and run in a reproducible environment.
- **docker-compose.yml** – orchestrates the Dash app and MongoDB together, making it easier to start the full dashboard stack with one command.
- **.dockerignore** – keeps unnecessary files out of the Docker image so the image remains smaller and cleaner.

Inside **scripts/**, you need to add:

- **scripts/seed_mongo.py** – is a one-time script used to seed MongoDB with the cleaned movie dataset so the dashboard can query data from the database.

Inside **notebook/**, you need to add:

- **notebook/lab13_dashboard.ipynb** – is the main submission notebook demonstrating the dashboard lab steps, including setup, MongoDB seeding, data access testing, callback behavior, Docker usage, and the final Assignment 13 solution.

Part 1 - Install Required Libraries and Tools

Dash runs on Flask and communicates with the browser using React under the hood. Install the following packages:

```
pip install dash dash-bootstrap-components pymongo gunicorn websockets
```

The table below explains each library's role:

Library	Version	Primary Role
dash	≥ 2.17	Core framework: Flask + React + Plotly wiring
dash-bootstrap-	≥ 1.6	Pre-built responsive grid, cards,

components		navbars
pymongo	≥ 4.6	Python driver for MongoDB
gunicorn	≥ 22.0	Production WSGI server (replaces Flask dev server)
websockets	≥ 12.0	Async WebSocket client for live data streams

Add these to requirements.txt file so the Dockerfile can install them during the image build.

```
dash>=2.17.0
dash-bootstrap-components>=1.6.0
pymongo>=4.6.0
gunicorn>=22.0.0
websockets>=12.0.0
```

[Part 2 - Understanding the Dash Framework](#)

Every Dash application follows the same basic flow and has exactly four parts. First, we **create the Dash app object**, which represents the main application. This is the starting point of the dashboard and it connects all other parts together. In production, the app can also expose its underlying Flask server so tools such as Gunicorn can run it properly.

After the app is created, the next step is to **define the layout**. The layout describes what the user will see in the browser. In simple words, it is the structure of the page. It can contain titles, text, dropdown menus, graphs, buttons, tables, and other visual elements. Dash translates this Python-based layout into HTML so it can be displayed as a web page.

The third part is **registering callbacks**. Callbacks are the interactive logic of a Dash app. They explain what should happen when the user interacts with the dashboard. For example, when a user selects a genre from a dropdown menu, a callback can automatically update a chart to show only movies from that selected genre. This is what makes the dashboard dynamic instead of just a static page.

The final part is **running the server**. This starts the Dash application and makes it available in the browser. During development, the app is usually run in debug mode because it automatically refreshes the browser when changes are saved. However, debug mode should only be used while developing the app and should not be used in production.

Part 3 - Setting Up the Dashboard Package

[3.1 Create src/dashboard/ __init__.py](#)

The `__init__.py` file makes the `src/dashboard/` folder a Python package. It allows the main dashboard functions, such as creating the layout and registering callbacks, to be imported easily in `app.py`.

[3.2 Create src/dashboard/data_access.py](#)

The `data_access.py` file contains all database-related logic for the dashboard. It connects to MongoDB, loads the movie dataset, and returns it as a Pandas DataFrame. If MongoDB is not available, it falls back to the cleaned CSV file from previous labs.

This file also provides helper functions for getting available genres, detecting the year range, and filtering movies by genre, year, or search text. Keeping this logic in one file makes the dashboard callbacks simpler, cleaner, and easier to test.

Part 4 - Building the Layout

This part creates the visual structure of the dashboard in `src/dashboard/layout.py`. The layout is separated from `app.py` so the main entry point stays clean and the page structure is easier to maintain and test.

[4.1 Create src/dashboard/layout.py](#)

The `layout.py` file defines what the user sees when the dashboard opens in the browser. It builds the full page using Dash and Bootstrap components, including the header, KPI cards, filters, charts, and live ticker section.

At the top of the file, the dataset is loaded once so the dashboard can prepare dropdown options and year slider values. The genre list is used for the genre filter, while the minimum and maximum years are used for the year range slider.

The layout includes a header with the dashboard title, a row of KPI cards showing summary metrics, and a filter bar where users can select a genre, choose a year range, or search for a movie title for example. Below the filters, the page contains multiple chart sections, such as top movies by revenue, rating distribution, budget versus revenue, movies released per year, and a simulated live data ticker.

A small helper function is also included to build the KPI cards automatically from the dataset. This keeps the layout code cleaner and avoids repeating the same card structure multiple times.

Part 5 - Implementing Callbacks

This part creates the callback logic in `src/dashboard/callbacks.py`. Callbacks are the most important part of a Dash dashboard because they make the page interactive. Whenever the user changes a filter, moves the year slider, types in the search field, or when the live timer updates, Dash automatically runs the correct callback function and updates the related chart.

[5.1 Create `src/dashboard/callbacks.py`](#)

The `callbacks.py` file contains all interactive behavior of the dashboard. It loads the movie dataset once and then uses that data inside the callback functions. The callbacks update the revenue chart, rating chart, budget versus revenue scatter plot, yearly movie trend chart, and live ticker chart.

The first group of callbacks reacts to the dashboard filters. When the user selects a genre, changes the year range, or searches for a movie title, the charts are updated using the filtered dataset. Each chart shows a different view of the data, such as top movies by revenue, rating distribution by genre, the relationship between budget and revenue, and the number of movies released per year.

The live ticker callback works differently. It updates automatically every few seconds using `dcc.Interval`. In this lab, the live data is simulated with random values, but the same idea could later be connected to a real streaming source or API.

The file also includes a helper function for empty charts. If there is no matching data or some required columns are missing, the dashboard shows a clean message instead of breaking.

5.2 Callback Patterns Used

The table below maps each callback in this lab to its pattern:

Pattern	Callback	Trigger	Output
$N \rightarrow 1$	Revenue bar	genre + year + search	one chart
$N \rightarrow 1$	Rating box	genre + year + search	one chart
$N \rightarrow 1$	Scatter	genre + year + search	one chart
$N \rightarrow 1$	Trend line	genre + year + search	one chart
$1 \rightarrow 1$	Live ticker	<code>dcc.Interval</code> tick	one chart

Part 6 - Creating the Application Entry Point

This part creates the main `app.py` file at the project root. This file is the starting point of the dashboard application and connects all previously created dashboard modules together.

[6.1 Create `app.py` at the Project Root](#)

The `app.py` file creates the Dash application, applies the Bootstrap theme, sets the page title, and prepares the app to run both locally and in production. It also makes the `src/` folder importable so the dashboard package can be used without installing it as a separate Python package. After the app is created, `app.py` loads the dashboard layout from `src/dashboard/layout.py` and registers all callbacks from `src/dashboard/callbacks.py`. This means the file does not contain the full layout or callback logic itself; it only connects the main parts of the application.

The file also exposes the underlying Flask server so the dashboard can be served by Gunicorn in production. When running locally, students can start the dashboard with `python app.py` and open it in the browser at `http://127.0.0.1:8050`. If everything works correctly, they should see the dark dashboard with filters, charts, KPI cards, and the live ticker updating every few seconds.

Part 7 - Seeding MongoDB

This part adds a script for loading the cleaned movie dataset into MongoDB. In a real production setup, the database would usually be filled by the full data pipeline. For local development, this script is used to quickly prepare MongoDB before running the dashboard.

[7.1 Create `scripts/seed\_mongo.py`](#)

The `seed_mongo.py` script reads the cleaned CSV file from `data/processed/`, connects to MongoDB, clears the existing movies collection, and inserts the movie records again. It also creates indexes for fields such as genre, year, and title so filtering and searching can be faster. This script only needs to be run when the database needs to be refreshed with the latest cleaned data.

Part 8 - Writing the Dockerfile

This part prepares the dashboard so it can run inside a Docker container. Docker makes the application easier to run on different machines because it packages the app, dependencies, and runtime environment together.

8.1 Create Dockerfile at the Project Root

The Dockerfile defines how the dashboard image is built. It starts from a lightweight Python image, installs the dependencies from requirements.txt, copies the project files into the container, and exposes port 8050, which is used by the Dash app. Instead of using the development server, the container runs the dashboard with Gunicorn, which is more suitable for production.

[8.2 Create .dockerignore](#)

The .dockerignore file tells Docker which files and folders should not be copied into the image. This keeps the image smaller and avoids including unnecessary files such as virtual environments, Git files, notebooks, logs, cache folders, and local environment files.

8.3 Build and Run the Container

After the Dockerfile is created, the image can be built and started as a container. When the container is running, the dashboard is available at <http://localhost:8050>. This confirms that the app can run independently inside Docker.

Part 9 - Docker Compose: App + MongoDB

This part uses Docker Compose to run both the Dash dashboard and MongoDB together. Instead of starting each container manually, Docker Compose defines the full stack in one configuration file and starts everything with one command.

[9.1 Create docker-compose.yml](#)

The docker-compose.yml file defines two services: the dashboard app and MongoDB. The dashboard service is built from the local Dockerfile, while the database service uses the official MongoDB image. Docker Compose also creates a shared network, so the app can connect to MongoDB using the service name db. The file also passes environment variables to the dashboard, such as the MongoDB connection string and database name. A named volume is used for MongoDB so the data can remain available even after containers are restarted.

9.2 Start the Full Stack

After the Compose file is ready, the full system can be started with Docker Compose. This builds the dashboard image, starts MongoDB, and runs the web app together.

9.3 Seed MongoDB from the Host

Once the stack is running, the seed script can be executed from the local machine. This fills the MongoDB database with the cleaned dataset so the dashboard can read and display the data.

9.4 Verify Everything Works

To verify the setup, open <http://localhost:8050> in the browser. The filters, charts, search box, year slider, and live ticker should all work. Changing the filters should immediately update the dashboard.

9.5 Stop the Stack

When finished, the stack can be stopped with Docker Compose. The containers can be stopped normally, or stopped together with the MongoDB volume if the stored database data should also be removed.

Part 10 - WebSocket Live Data Integration

This part explains an advanced option for dashboards that need real live updates. In this lab, the live ticker uses `dcc.Interval`, which refreshes the chart every few seconds. This is simple and enough for most dashboards.

WebSockets are used when the dashboard needs faster, server-pushed updates, for example when new data must appear almost instantly. They are more complex because they require a background listener and asynchronous communication. In this lab, WebSockets are only shown as a reference pattern, while the implemented dashboard continues to use the simpler interval-based ticker.

10.1 How `dcc.Interval` Compares to WebSockets

`dcc.Interval` works by checking for new data repeatedly after a fixed time interval. It is easier to implement and is suitable when updates every few seconds are acceptable. WebSockets work differently because the server pushes new data to the app as soon as it becomes available. This is better for real-time systems, but it requires more setup and careful memory handling.

10.2 WebSocket Client Sketch

The WebSocket example shows how a background client could listen to a live data stream and store incoming messages in a shared buffer. The dashboard callback would then read from that

buffer instead of generating random demo values. This pattern is useful if the dashboard is later connected to a real live data source.

Part 11 - Integrate into run_pipeline.py

This part updates the existing pipeline file so it also gives instructions for starting the dashboard. The dashboard is not started automatically because it is a long-running web server, unlike normal pipeline steps that run and finish. Instead, the pipeline prints a short dashboard-ready message with the commands for starting the app locally or with Docker. This makes the dashboard part of the project workflow while still keeping it separate from the automated data pipeline.

Assignment Task:

1. Dash Application (35%)
 - Create app.py with a full Dash layout using html , dcc , and dash-bootstrap-components.
 - Include at least three interactive components
 - Display at least four charts that all respond to the same filters.
2. Callbacks (25%)
 - Implement at least five callbacks.
 - Each callback must have at least one Input and one Output.
 - Include the live-updating ticker that uses dcc.Interval.
 - The revenue bar chart callback and one additional chart callback.
3. MongoDB Integration (15%)
 - The data_access.py module must be read from MongoDB
 - Fall back to the cleaned CSV gracefully if MongoDB is unavailable.
 - The seed script must successfully populate the collection.
4. Docker Deployment (15%)
 - Dockerfile must use python:3.12-slim base and Gunicorn as the CMD.
 - docker-compose.yml must define web and db services.
 - docker compose up --build must start the full stack without errors.
 - Include a .dockerignore to keep the image lean.
5. Deployment Documentation (10%)
 - In your notebook (notebook/lab13_dashboard.ipynb) include a Markdown cell after the deployment section that documents the deployment process. It must cover:
 - Prerequisites (Docker Desktop installed, ports available)
 - The exact sequence of commands to start the stack and seed the database
 - How to verify the app is running (URL, what to look for)
 - How to stop the stack cleanly
 - At least one troubleshooting tip (e.g., what to do if port 8050 is in use)
6. **OPTIONAL-BONUS TASK:** Deploy application and paste link in README file in github repository